

Anonymous Verification Protocol Details

▷ For each verifier i to register and initialize

AnonyReg($1^\lambda, pp$) $\rightarrow (C_i, UKey_i, \pi_{\text{mix}})$

- generate key-pair:
- set $ok_i \xleftarrow{\$} \{0, 1\}^\lambda$, $sk_i = H_1(ok_i)$, $pk_i = H_1(sk_i)$;
- set $UKey_i = (sk_i, pk_i)$;
- obtain deposit from mixer and submit transaction via relayer;
// hide verifier's identity information
- commit stake note: $C_i = r_i \mathbf{G} + v_i \mathbf{H}$;
// r_i is verifier randomness, v_i is the staked amount
- generate ZK-membership proof:
 $\pi_{\text{mix}} = \text{NIZK}(\exists v_i, r_i : C_i = r_i \mathbf{G} + v_i \mathbf{H})$
- return $(C_i, UKey_i, \pi_{\text{mix}})$

▷ Each verifier i stakes to participate in the election

AnonyStake($C_i, pp, UKey_i$) $\rightarrow (\pi_{N_i}, C_{\text{stake}}, C_{\text{change}}, C_{\text{receipt}}, \Delta C)$

- parse pp as $(v_{\text{stake}}, \mathcal{A}_\tau)$;
- compute stake note: $C_{\text{stake}} = r_{\text{stake}} \mathbf{G} + v_{\text{stake}} \mathbf{H}$;
- return $r_{\text{stake}} \mathbf{G}$ as commitment of randomness;
- compute change note: $C_{\text{change}} = r_{\text{change}} \mathbf{G} + v_{\text{change}} \mathbf{H}$;
- compute balance adjustment: $\Delta C = r' \mathbf{G}$;
- derive double-spending tag: $N_i = H_1(r_i, C_i, \text{index}(C_i))$;

- generate ZK-proof of tag correctness:

$\pi_{N_i} = \text{NIZK}(\exists r_i : N_i = H_1(r_i, C_i, \text{index}(C_i)))$;

- return $(\pi_{N_i}, C_{\text{stake}}, C_{\text{change}}, \Delta C)$.

▷ For each verifier V_i in system to verify τ

Endorse($pp, st, C_{\text{stake}}, H_{\text{root}})$ $\rightarrow Res$

- generate the proof to interact with blockchain;
- compute Merkle-tree path proof π_{path} ;
- compute linkability proof:
 $\pi_l = \text{NIZK}\{\exists r_{\text{stake}} : C_{\text{stake}} = r_{\text{stake}} \mathbf{G} + v_{\text{stake}} \mathbf{H}\}$;
- **If some $\sigma_k \in \tau$ is erroneous:**
 - record σ_k ; *// error snapshot triggers challenge*
 - set $Res \leftarrow \text{VerEndorse}(0, \sigma_k, \pi_{\text{path}}, \pi_l)$
- **If τ is correct:**
 - record each $\{\sigma_l \in \tau_k^t\}$ Gas;
// execution weight of intermediate snapshots
 - compute workload commitment:
 $Gas_i = H_1(\text{Ser}\{\sigma_k, \dots, \sigma_t\}, r_1)$, where r_1 is verifier randomness;
 - set $Res \leftarrow \text{VerEndorse}(1, Gas_i, \pi_l, \pi_{\text{path}})$
- **Return Res**